Group Members:

Oman Shahid (22L-6556)

Aown Raza (22L-6547)

Ahmed Mukhtar (22L-8225)

1. Project Introduction

The **BZip2-Impl** project is a high-performance compression utility developed for the **FAST School of Computing**. The primary objective was to implement a lossless compression engine capable of handling large-scale data blocks while maintaining industry-leading compression ratios. By utilizing a multi-stage transformation pipeline, the system effectively prepares data for entropy coding, resulting in massive reductions in file size.

2. Technical Architecture: The Pipeline

The engine follows a sequential pipeline where each stage reduces the entropy of the data to facilitate higher compression in the final step.

Stage 1: Threshold-Based RLE (RLE1)

To prevent the "data expansion" common in standard Run-Length Encoding, this stage uses a **threshold of 4**. It only encodes sequences where a character repeats at least four times.

- **Efficiency Gain:** This minimizes metadata overhead and ensures that non-repetitive data remains untouched before hitting the BWT stage.

Stage 2: Burrows-Wheeler Transform (BWT)

The BWT is the core organizational stage. It rearranges the input string into a permutation that groups similar characters together. This does not compress the data itself but makes it highly predictable for subsequent stages.

Stage 3: Move-To-Front (MTF)

MTF takes the clustered output of the BWT and transforms it into a series of indices. Because similar characters are grouped, the MTF stage produces a stream heavily biased toward the value **0**.

Stage 4: Zero-Focused RLE (RLE2)

Since the MTF output is "zero-heavy," RLE2 collapses runs of zeros into simple [Tag][Length] pairs. This significantly reduces the volume of data passed to the entropy coder.

Stage 5: Asymmetric Numeral Systems (rANS)

The final stage is the entropy coder. Unlike Huffman coding, which is restricted to whole-bit boundaries, rANS allows for **fractional bit representation**, squeezing the data down to its theoretical limit.

---

3. Advanced Optimizations (Bonus Features)

To meet the "Grand Challenge" criteria and secure the 10% bonus marks, three critical optimizations were implemented:

3.1 Suffix Array Construction

Standard BWT implementations use a matrix sort, which is unusable for blocks larger than a few kilobytes. I implemented a **Suffix Array (Prefix Doubling)** algorithm.

- **Impact:** This reduced the time complexity of the BWT to , allowing the system to process 500,000-byte blocks in seconds rather than minutes.

3.2 rANS Entropy Engine

By choosing rANS over traditional Huffman or Arithmetic coding, the project achieves a superior balance between speed and compression ratio.

- **Impact:** rANS is significantly faster than Arithmetic coding while providing nearly identical compression ratios, contributing to our **414x** peak ratio.

3.3 Lossless Pipeline Synchronization

A critical engineering challenge was ensuring data integrity across stage boundaries. I implemented a custom **Length-Sync Logic** in the main pipeline.

- **Impact:** This solved a "tail-end" truncation bug, ensuring that every byte (including trailing strings like " SHAH") is perfectly restored during decompression.

---

4. Performance Metrics

The following results were captured using a block size of **500,000 bytes**:

Metric	Repetitive Text (.txt)	Patterned Binary (.bin)
Original Size	15.73 MB	10.48 MB
Compression Ratio	**414.04x**	**275.35x**
Encoding Time	9.40 Seconds	6.17 Seconds
Memory Footprint	28.80 MB	22.98 MB

Export to Sheets

---

5. Conclusion

The **BZip2-Impl** successfully bridges the gap between theoretical data compression and practical, high-performance software. By combining modern entropy coding (rANS) with optimized string sorting (Suffix Arrays), the project delivers professional-grade results. The implementation is 100% lossless, stable, and fulfills all requirements for the semester's advanced bonus marks.